

文章笔记

5 Agent Skill Design Patterns Every ADK Developer Should Know

基于公开课程资料整理

2026-03-17

作者/来源: Google Cloud Tech

发布日期: 2026-03-17

内容形式: X Article

原文链接: <https://x.com/GoogleCloudTech/status/2033953579824758855>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：从格式到内容设计	2
1.1 本章小结	2
2 Pattern 1: Tool Wrapper (工具封装)	2
2.1 工作原理	3
2.2 适用场景	3
2.3 本章小结	3
3 Pattern 2: Generator (生成器)	3
3.1 工作原理	3
3.2 适用场景	4
3.3 本章小结	4
4 Pattern 3: Reviewer (审查者)	4
4.1 工作原理	4
4.2 适用场景	5
4.3 本章小结	5
5 Pattern 4: Inversion (反转)	5
5.1 工作原理	5
5.2 适用场景	6
5.3 本章小结	6
6 Pattern 5: Pipeline (流水线)	6
6.1 工作原理	6
6.2 适用场景	6
6.3 本章小结	7
7 如何选择合适的模式	7
7.1 本章小结	7
8 总结与延伸	7
8.1 核心要点回顾	7
8.2 延伸思考	8
8.3 拓展阅读	8

1 引言：从格式到内容设计

随着 Agent 工具生态的快速发展, 超过 30 种 Agent 工具 (如 Claude Code、Gemini CLI、Cursor 等) 已经在 SKILL.md 的文件格式上达成了标准化共识。格式问题基本已经解决, 真正的挑战转移到了内容设计层面。

什么是 SKILL.md ?

SKILL.md 是 Agent Skills 规范中的核心文件, 用于定义一个 skill 的指令、触发条件和行为逻辑。它是 Agent 工具加载和执行 skill 的入口。该规范由 Google ADK (Agent Development Kit) 推动, 已被多个主流 Agent 工具采纳。

规范本身只说明了如何**打包**一个 skill, 却没有提供如何**设计内部逻辑**的指导。例如, 一个封装 FastAPI 约定的 skill 和一个四步文档生成 pipeline, 虽然外部的 SKILL.md 文件看起来完全一样, 但内部运作方式截然不同。

核心观点

通过研究生态系统中 skill 的构建方式 (从 Anthropic 的仓库到 Vercel 和 Google 的内部指南), 可以总结出五种反复出现的设计模式, 帮助开发者构建更可靠的 Agent。

本文介绍的五种模式分别是:

1. **Tool Wrapper**: 让 Agent 成为任意库的即时专家
2. **Generator**: 基于可复用模板生成结构化文档
3. **Reviewer**: 按严重程度对代码进行检查评分
4. **Inversion**: Agent 先采访你, 再行动
5. **Pipeline**: 强制执行带检查点的多步骤 workflow

1.1 本章小结

Agent Skill 的格式标准化已经成熟, 开发者的关注点应从”如何写对 YAML”转向”如何设计 skill 内部的逻辑结构”。五种设计模式为这一问题提供了系统性的解决框架。

2 Pattern 1: Tool Wrapper (工具封装)

Tool Wrapper 为 Agent 提供针对特定库的**按需上下文**。与其将 API 约定硬编码到 system prompt 中, 不如将它们打包成一个 skill, Agent 只在实际使用该技术时才加载相关上下文。

Tool Wrapper 的核心机制

SKILL.md 文件监听用户 prompt 中的特定库关键词, 动态从 **references/** 目录加载内部文档, 并将这些规则作为绝对真理应用。这正是将团队内部编码规范或特定框架最佳实践直接分发到开发者 workflow 中的机制。

2.1 工作原理

Tool Wrapper 是最简单的模式。其核心流程为：

1. Agent 检测到用户 prompt 中包含特定库的关键词（如 “FastAPI”）
2. SKILL.md 指令触发，动态加载 references/ 目录下的约定文档（如 conventions.md）
3. Agent 将加载的规则作为上下文，指导代码编写或审查

2.2 适用场景

- 分发团队内部的编码规范
- 封装特定框架的最佳实践
- 在不同项目间复用一致的技术标准

按需加载的优势

Tool Wrapper 不会在所有对话中都占用 context window。只有当用户的请求涉及特定技术栈时，相关文档才会被加载。这种按需加载策略既节省了 token 开销，又保证了上下文的精准性。

2.3 本章小结

Tool Wrapper 是最简单也最基础的模式，通过关键词触发和按需加载，将特定库的知识动态注入 Agent 的上下文中，适合封装编码规范和框架约定。

3 Pattern 2: Generator（生成器）

Tool Wrapper 负责应用知识，而 Generator 则负责强制输出一致性。如果你发现 Agent 每次运行时生成的文档结构都不同，Generator 通过编排一个填空式流程来解决这个问题。

Generator 的关键目录结构

Generator 利用两个可选目录：

- assets/：存放输出模板
- references/：存放风格指南

SKILL.md 中的指令充当项目经理角色，告诉 Agent 加载模板、读取风格指南、向用户询问缺失变量，然后填充文档。

3.1 工作原理

Generator 的执行流程是严格有序的：

1. 加载 assets/ 中的输出模板
2. 读取 references/ 中的风格指南

3. 向用户询问缺失的变量
4. 按照模板和风格指南填充文档

关键在于: `SKILL.md` 文件本身不包含实际的布局或语法规则, 它只负责**协调资产的检索**并强制 Agent 按步骤执行。

3.2 适用场景

- 生成可预测的 API 文档
- 标准化 commit message
- 脚手架式的项目架构搭建

常见误区

不要把模板内容直接写在 `SKILL.md` 中。Generator 模式的精髓在于**关注点分离**: 指令文件负责流程编排, 模板和风格指南作为独立资产存储在各自的目录中。这样更换模板或风格指南时不需要修改 skill 逻辑。

3.3 本章小结

Generator 通过模板 + 风格指南 + 填空流程的组合, 确保 Agent 每次生成的文档结构一致。指令与资产的分离使得 skill 具有高度可复用性。

4 Pattern 3: Reviewer (审查者)

Reviewer 模式将**检查什么**与**如何检查**分离。与其在 system prompt 中写一长串代码质量规则, 不如将模块化的评分标准 (rubric) 存储在 `references/review-checklist.md` 文件中。

Reviewer 的核心设计

指令保持静态不变, Agent 动态加载外部 checklist 中的具体审查标准, 并按严重级别输出结构化的评审结果。更换 checklist 即可切换审查维度: Python 风格检查 → OWASP 安全审计, 无需修改 skill 基础设施。

4.1 工作原理

1. 用户提交代码
2. Agent 加载 `references/review-checklist.md` 中的检查清单
3. Agent 逐项对照清单进行系统化评审
4. 按严重级别 (severity) 分组输出发现的问题

4.2 适用场景

- 自动化 PR review
- 安全漏洞预检
- 代码质量评估
- 合规性检查

模块化的威力

Reviewer 模式最大的优势在于 checklist 的可插拔性。同一套 skill 基础设施，配合不同的 checklist 文件，可以变成完全不同的专业审查工具。这种设计让 skill 的复用效率最大化。

4.3 本章小结

Reviewer 通过将审查标准外置为独立的 checklist 文件，实现了检查逻辑与检查内容的解耦。更换 checklist 即可获得全新的审查能力，是自动化代码审查和安全审计的理想选择。

5 Pattern 4: Inversion (反转)

Agent 天生倾向于**猜测并立即生成**。Inversion 模式翻转了这一动态：不是用户驱动 prompt、Agent 执行，而是 **Agent 充当采访者**，在行动之前先收集完整的上下文。

Inversion 的核心约束

Inversion 依赖于明确的、不可协商的门控指令 (gating instructions)，例如”在所有阶段完成之前，不得开始构建”。Agent 按顺序提出结构化问题，等待用户回答后才进入下一阶段，直到获得完整的需求和部署约束后才生成最终输出。

5.1 工作原理

1. Agent 按预定义的阶段依次向用户提问
2. 每个阶段收集特定类别的信息（需求、约束、偏好等）
3. 严格的门控条件阻止 Agent 跳过任何阶段
4. 只有所有答案收集完毕后，Agent 才综合生成最终输出

为什么需要显式门控？

如果没有显式的门控指令，Agent 很容易在收集到部分信息后就急于生成结果。这会导致输出质量低下，遗漏关键需求。门控指令（如 “DO NOT start building until all phases are complete”）是 Inversion 模式的生命线，必须用强硬、明确的语言表达。

5.2 适用场景

- 项目规划（需要全面了解需求后才能制定方案）
- 架构设计（需要了解各种约束条件）
- 复杂任务分解（需要明确上下文后才能合理拆分）

5.3 本章小结

Inversion 模式通过显式门控指令，强制 Agent 在行动前完成信息收集。这种“先问后做”的策略显著提升了输出质量，特别适合需要全面上下文的复杂任务。

6 Pattern 5: Pipeline（流水线）

对于复杂任务，跳过步骤或忽略指令的代价是不可接受的。Pipeline 模式通过**硬性检查点**（hard checkpoints）强制执行严格的顺序工作流。

Pipeline 的核心特征

指令本身即是工作流定义。通过实现显式的菱形门控条件（diamond gate conditions），例如要求用户在 docstring 生成步骤确认后才能进入最终组装阶段，Pipeline 确保 Agent 无法绕过复杂任务直接呈现未经验证的最终结果。

6.1 工作原理

1. 工作流被分解为多个严格有序的步骤
2. 每个步骤之间设置门控条件（通常需要用户确认）
3. 不同步骤按需加载不同的 reference 文件和模板
4. Agent 被明确禁止跳过任何步骤或门控

上下文窗口管理

Pipeline 模式的一个精妙之处在于，它利用所有可选目录，但只在需要特定资产的步骤才加载对应的 reference 文件和模板。这种按步骤加载的策略保持了 context window 的整洁，避免了不必要的 token 消耗。

6.2 适用场景

- 多步骤文档生成流水线
- 需要人工审批的自动化工作流
- 质量要求严格的代码生成任务

6.3 本章小结

Pipeline 模式是五种模式中最严格的一种，通过硬性检查点和门控条件确保复杂工作流的每一步都被正确执行和验证。适合对质量和完整性要求极高的场景。

7 如何选择合适的模式

每种模式回答的是不同的问题。以下是选择指南：

模式	核心问题	典型场景
Tool Wrapper	如何注入特定库的知识？	编码规范、框架约定
Generator	如何保证输出结构一致？	API 文档、项目脚手架
Reviewer	如何系统化检查代码？	PR review、安全审计
Inversion	如何确保信息收集完整？	项目规划、架构设计
Pipeline	如何强制多步骤顺序执行？	文档流水线、审批 workflow

表 1: 五种 Agent Skill 设计模式对比

模式可以组合

这五种模式并非互斥。它们可以自由组合：

- Pipeline 可以在最后一步嵌入 Reviewer 来自检工作成果
- Generator 可以在开始阶段使用 Inversion 来收集必要的模板变量

得益于 ADK 的 SkillToolset 和渐进式披露（progressive disclosure）机制，Agent 在运行时只在需要的模式上花费 context token。

7.1 本章小结

选择模式时应根据核心问题匹配，而非盲目套用。更重要的是，模式之间可以组合使用，形成更强大的复合 skill。

8 总结与延伸

8.1 核心要点回顾

本文系统梳理了 Agent Skill 的五种设计模式：

1. **Tool Wrapper**：按需加载，注入特定库知识
2. **Generator**：模板驱动，确保输出一致性
3. **Reviewer**：checklist 外置，实现可插拔审查
4. **Inversion**：先问后做，门控式信息收集
5. **Pipeline**：硬性检查点，强制顺序执行

最重要的启示

不要试图将复杂而脆弱的指令塞进一个 system prompt 中。将 workflow 分解，应用合适的结构化模式，才能构建出可靠的 Agent。

8.2 延伸思考

- **Agent Skills 规范**是开源的，且已被 ADK 原生支持。开发者可以直接基于这些模式开始构建。
- 随着更多 Agent 工具采纳统一的 skill 格式，**跨工具的 skill 复用**将成为可能。一个设计良好的 skill 可以同时使用在 Claude Code、Gemini CLI 和 Cursor 中。
- 模式的组合潜力值得进一步探索：例如 Inversion + Generator + Reviewer 的三层组合，可以构建一个“先收集需求、再生成文档、最后自审”的完整 workflow。

8.3 拓展阅读

- Google Agent Development Kit (ADK) 官方文档
- Agent Skills 开源规范
- 原文作者：@Saboo_Shubham_ 和 @lavinigam